



TECNOLÓGICO
NACIONAL DE MÉXICO



Tecnológico Nacional de México

Instituto Tecnológico de Culiacán

Tema:

“App detección de rostros con Visión Artificial - **Reconoce-Tec**”

Tópicos de Inteligencia Artificial

Profesor:

Mora Felix Zuriel Dathan

Integrantes:

Corrales Cabanillas Eleazar

Trujillo Acosta Bryant

[Repositorio](#)

Índice

1. Introducción	3
2. Objetivo General	4
2.1 Objetivos Específicos	4
3. Descripción General	5
4. Construcción del Dataset	6
5. Desarrollo	7
5.1 Procesamiento y Preparación de los Datos	7
5.2 Investigación y Selección de la Arquitectura Neuronal	9
5.3 Configuración y entrenamiento del Modelo	10
5.4 Resultados y Validación del Modelo	11
5.5 Desarrollo e Implementación de la Aplicación Móvil	13
5.6 Capturas de Pantalla de Ejecución	14
5.6.1 Interfaz del Móvil	14
5.6.2 Pantalla de carga con el nombre de la App y Slogan	15
5.6.3 Login	16
5.6.4 Panel principal de la Aplicación	17
5.6.5 Panel de control de alumnos	18
5.6.6 Reconocedor de Rostros	19
5.6.7 Pruebas	20
5.6.8 Registro de asistencia	21
6. Resumen de Tecnologías Utilizadas	22
7. Conclusiones	23
Referencias	24

1. Introducción

La visión artificial es una rama de la inteligencia artificial que permite a las computadoras analizar e interpretar imágenes y videos para extraer información útil. Gracias a los avances en Machine Learning y Deep Learning, actualmente es posible desarrollar sistemas capaces de identificar personas, objetos y patrones visuales con un alto nivel de precisión. Una de las aplicaciones más utilizadas de esta tecnología es el reconocimiento facial, empleado en sistemas de seguridad, control de acceso y registro de asistencia.

El presente proyecto tuvo como objetivo desarrollar una aplicación móvil capaz de reconocer estudiantes mediante el análisis de sus rostros. Para ello, fue necesario realizar un proceso completo que incluyó la construcción de un dataset, la investigación de diferentes arquitecturas neuronales, el entrenamiento y validación de un modelo de inteligencia artificial, así como su integración en una aplicación móvil funcional.

La construcción del dataset se realizó a partir de videos de compañeros y amigos del Instituto Tecnológico de Culiacán. A partir de estos videos se extrajeron fotogramas representativos y posteriormente se aplicaron técnicas de detección facial para obtener imágenes centradas únicamente en los rostros. Además, se incorporaron variaciones de iluminación, ángulos y expresiones faciales mediante técnicas de aumento de datos, con el propósito de mejorar la capacidad de generalización del modelo.

Posteriormente, se investigaron distintas arquitecturas utilizadas en reconocimiento facial, entre ellas FaceNet, YOLO Face y **MobileNetV2**. Tras analizar aspectos como precisión, velocidad de inferencia, complejidad de implementación y recursos computacionales requeridos, se seleccionó **MobileNetV2** por su eficiencia y facilidad de integración en dispositivos móviles mediante TensorFlow Lite.

Una vez seleccionada la arquitectura, se realizó el entrenamiento utilizando TensorFlow y Google Colab, aplicando procesos de normalización, segmentación de datos y ajuste de hiperparámetros. El desempeño del modelo fue evaluado mediante métricas como accuracy, precision, recall y F1-score, obteniendo resultados satisfactorios para el contexto del proyecto.

Finalmente, el modelo entrenado fue integrado en una aplicación desarrollada con Flutter, la cual permite detectar rostros, identificar estudiantes, consultar información académica almacenada en una base de datos SQLite y registrar asistencias automáticamente. De esta manera, el proyecto integra conocimientos de visión artificial, aprendizaje automático, desarrollo móvil y bases de datos en una solución orientada al ámbito educativo y a la validación de los conocimientos adquiridos durante el curso.

2. Objetivo General

- Desarrollar una aplicación móvil de reconocimiento facial basada en técnicas de visión artificial y Machine Learning, capaz de identificar estudiantes a partir de imágenes de sus rostros y mostrar su información académica básica. Asimismo, se busca validar la utilidad de la solución mediante un caso de uso práctico enfocado en el registro de asistencia, utilizando una base de datos local para almacenar y gestionar la información correspondiente. Todo ello integrando un modelo de inteligencia artificial entrenado y optimizado para su ejecución eficiente en dispositivos móviles.

Nota: La integración del módulo de asistencia tiene como justificación poner en practica el proyecto en un área de aplicación que nos permita validar el valor que tienen soluciones innovadoras como la que fue desarrollada en el proyecto.

2.1 Objetivos Específicos

- **Construcción y aumento del dataset:** Reúne un conjunto de imágenes de rostros de estudiantes. Amplía este dataset capturando imágenes bajo diferentes condiciones de iluminación, ángulos y expresiones faciales. Asegúrate de que el dataset sea variado y representativo para mejorar la precisión del modelo.
- **Investigación de arquitecturas neuronales:** Investiga diferentes arquitecturas de redes neuronales especializadas en detección de rostros. Justifica cuál es la más adecuada para el contexto educativo, considerando precisión, velocidad y recursos computacionales. Analiza proyectos similares y sus resultados para fundamentar tu elección.
- **Procesamiento y entrenamiento del modelo:** Prepara el dataset realizando tareas como normalización, segmentación y aumento de datos. Entrena el modelo ajustando parámetros como tasa de aprendizaje, número de capas y épocas, para optimizar el desempeño en la detección de rostros.
- **Evaluación del modelo:** Utiliza métricas relevantes como precisión, recall y F1-score para evaluar el desempeño del modelo. Compara los resultados obtenidos con los de otras arquitecturas y justifica posibles ajustes para mejorar la eficacia.
- **Desarrollo de la interfaz de usuario:** Diseña una interfaz intuitiva y atractiva, compatible con Android e iOS. Piensa en la disposición de la información para que sea clara, accesible y fácil de navegar para los usuarios. Considera la experiencia del usuario al interactuar con la app y cómo se visualiza la información de los estudiantes.
- **Implementación de la base de datos:** Crea una base de datos eficiente que permita consultar y actualizar información de los estudiantes de manera rápida y segura. Asegúrate de que los mecanismos de acceso y modificación sean confiables y fáciles de usar.

3. Descripción General

El sistema desarrollado consiste en una aplicación móvil de reconocimiento facial orientada al ámbito educativo. Su propósito es identificar estudiantes mediante el análisis de imágenes capturadas por la cámara del dispositivo móvil y mostrar información académica almacenada en una base de datos local.

El funcionamiento general del sistema inicia con la autenticación de un administrador, quien tiene acceso a las funcionalidades principales de la aplicación. Una vez dentro del sistema, puede acceder al módulo de reconocimiento facial o al panel de control para administrar la información de los estudiantes registrados.

Cuando se utiliza el módulo de reconocimiento, la aplicación captura una imagen mediante la cámara del dispositivo y emplea técnicas de visión artificial para detectar la presencia de un rostro. Posteriormente, la imagen es procesada y enviada a un modelo de inteligencia artificial previamente entrenado, el cual realiza la clasificación y determina la identidad del estudiante. Una vez identificado, la aplicación consulta una base de datos SQLite para recuperar información asociada al estudiante, incluyendo su nombre, semestre y carrera.

Como caso de uso práctico, el sistema incorpora un módulo de registro de asistencia que almacena automáticamente la fecha, hora y nivel de confianza de cada reconocimiento exitoso. Además, se desarrolló un panel de control que permite consultar, modificar y eliminar registros de estudiantes, proporcionando una solución integral para la gestión y consulta de información académica.

La aplicación fue desarrollada utilizando Flutter para la interfaz móvil, TensorFlow Lite para la ejecución del modelo de inteligencia artificial en el dispositivo, MediaPipe y Google ML Kit para la detección facial, y SQLite para el almacenamiento local de datos.

4. Construcción del Dataset

La calidad de un sistema de reconocimiento facial depende en gran medida de la calidad y diversidad de los datos utilizados durante el entrenamiento. Por esta razón, una de las etapas más importantes del proyecto consistió en la construcción de un dataset representativo que permitiera al modelo aprender las características faciales de cada estudiante.

Para la creación del conjunto de datos se recopilaron videos de compañeros y amigos del Instituto Tecnológico de Culiacán. Cada video fue nombrado utilizando el número de control del estudiante correspondiente, lo que permitió utilizar dicho identificador como etiqueta de clasificación durante el entrenamiento del modelo. En total se trabajó con siete estudiantes diferentes, representando siete clases dentro del problema de reconocimiento facial.

A partir de los videos se realizó una extracción automática de fotogramas utilizando OpenCV. Este procedimiento permitió obtener múltiples imágenes de cada estudiante sin necesidad de realizar capturas manuales individuales. Posteriormente, se empleó MediaPipe Face Detection para detectar y recortar únicamente la región correspondiente al rostro, eliminando información irrelevante del fondo y mejorando la calidad de los datos utilizados para el entrenamiento.

Durante la captura de los videos se procuró incluir diferentes condiciones de iluminación, expresiones faciales, distancias y ángulos de visión. Adicionalmente, se aplicaron técnicas de aumento de datos (Data Augmentation) y procesos de limpieza para descartar imágenes sin rostros detectables o de baja calidad. Finalmente, se realizó un balanceo de clases para evitar que algunos estudiantes tuvieran una representación excesiva dentro del dataset, reduciendo así posibles sesgos durante el entrenamiento.

El resultado fue un conjunto de datos compuesto por imágenes faciales previamente procesadas y etiquetadas, listo para ser utilizado en las etapas de entrenamiento y validación del modelo de inteligencia artificial.

Es importante mencionar que el dataset utilizado fue relativamente pequeño en comparación con los conjuntos de datos empleados en sistemas comerciales de reconocimiento facial, los cuales suelen contener miles o incluso millones de imágenes. Sin embargo, para los objetivos académicos del proyecto, el conjunto de datos resultó suficiente para demostrar el proceso completo de construcción de un sistema de visión artificial, desde la recopilación y preparación de datos hasta el entrenamiento, validación e integración del modelo en una aplicación móvil funcional.

Asimismo, este enfoque permitió comprender la importancia que tienen la calidad, diversidad y balance de los datos en el desempeño final de un modelo de inteligencia artificial. Aunque el sistema obtuvo resultados satisfactorios dentro de las condiciones evaluadas, una posible mejora futura consistiría en ampliar el número de estudiantes y la cantidad de imágenes por clase, lo que permitiría incrementar la capacidad predictiva del modelo y acercarlo a escenarios de uso más cercanos a una implementación real.

5. Desarrollo

5.1 Procesamiento y Preparación de los Datos

Una vez recopilados los videos de los estudiantes, fue necesario realizar una serie de procesos para transformar la información en un conjunto de datos adecuado para el entrenamiento del modelo de inteligencia artificial. Esta etapa tuvo como objetivo garantizar que las imágenes utilizadas fueran consistentes, representativas y estuvieran enfocadas únicamente en la información relevante para la tarea de reconocimiento facial.

Como primer paso, se utilizó OpenCV para extraer fotogramas de cada video. Este procedimiento permitió obtener múltiples imágenes de cada estudiante a partir de una sola grabación, incrementando la cantidad de ejemplos disponibles para el entrenamiento. La extracción se realizó de forma automática tomando fotogramas a intervalos determinados, con el fin de capturar variaciones naturales en la posición del rostro y las expresiones faciales.

```
##extraer frames desde videos

# para videos cortos
FRAME_STEP = 5
MAX_FRAMES_PER_VIDEO = 80

limpiar_carpeta(FRAMES_DIR)

total_frames = 0
frames_por_clase = {}

for video_path in videos:
    clase = video_path.stem
    salida_clase = FRAMES_DIR / clase
    salida_clase.mkdir(parents=True, exist_ok=True)

    cap = cv2.VideoCapture(str(video_path))

    frame_idx = 0
    guardados = 0

    while True:
        ok, frame = cap.read()

        if not ok:
            break

        if frame_idx % FRAME_STEP == 0:
            nombre_frame = f"{clase}_{frame_idx:06d}.jpg"
            cv2.imwrite(str(salida_clase / nombre_frame), frame)

            guardados += 1
            total_frames += 1

            if guardados >= MAX_FRAMES_PER_VIDEO:
                break

        frame_idx += 1

    cap.release()

    frames_por_clase[clase] = guardados
    print(f"{clase}: {guardados} frames")

print("\ntotal de frames extraidos:", total_frames)

if total_frames == 0:
    raise RuntimeError("No se extrajo ningún frame. Revisa VIDEO_DIR y los formatos de video.")

28170638: 68 frames
28170639: 34 frames
21170495: 46 frames
21170937: 32 frames
22170618: 30 frames
22170621: 80 frames
22170640: 73 frames
```

–Img 1. Fragmento de código utilizado

Posteriormente, se empleó MediaPipe Face Detection para detectar y localizar los rostros presentes en cada imagen. Una vez detectado el rostro, se realizó un recorte de la región facial, eliminando gran parte del fondo y otros elementos que no aportaban información relevante para la clasificación. Este proceso permitió centrar la atención del modelo exclusivamente en las características faciales de cada estudiante.

Rostros 20170638: 100%	68/68	[00:13<00:00, 5.16it/s]
Rostros 20170639: 100%	34/34	[00:04<00:00, 7.27it/s]
Rostros 21170495: 100%	46/46	[00:09<00:00, 4.61it/s]
Rostros 21170937: 100%	32/32	[00:06<00:00, 4.80it/s]
Rostros 22170618: 100%	30/30	[00:04<00:00, 6.30it/s]
Rostros 22170621: 100%	80/80	[00:09<00:00, 8.37it/s]
Rostros 22170640: 100%	73/73	[00:11<00:00, 6.59it/s]
Rostros guardados: 291		
Frames sin rostro detectado: 72		

– Img 2. Ejecución del script en Google Colab

Las imágenes recortadas fueron redimensionadas a una resolución de 160×160 píxeles, tamaño compatible con la arquitectura seleccionada para el entrenamiento. Además, se aplicaron procesos de normalización para escalar los valores de los píxeles a rangos adecuados para la red neuronal, favoreciendo una convergencia más estable durante el aprendizaje.



–Img 3. Normalización y Detección de Rostros

Con el fin de evitar sesgos en el entrenamiento, también se realizó un balanceo del dataset, limitando la cantidad de imágenes por estudiante para que todas las clases tuvieran una representación similar. Finalmente, el conjunto de datos fue dividido en subconjuntos de entrenamiento y validación, permitiendo evaluar el desempeño del modelo sobre imágenes que no habían sido utilizadas durante el proceso de aprendizaje.

Estas etapas de procesamiento y preparación resultaron fundamentales para mejorar la calidad de los datos y contribuir a la obtención de un modelo más robusto y capaz de generalizar adecuadamente ante nuevas imágenes.

5.2 Investigación y Selección de la Arquitectura Neuronal

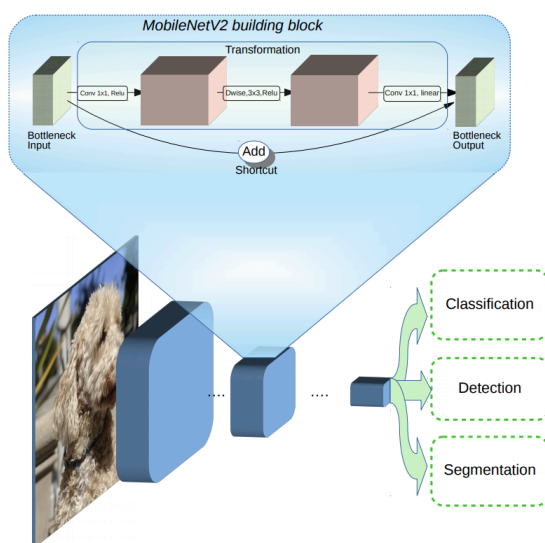
Entre las arquitecturas analizadas se encuentra FaceNet, una de las soluciones más reconocidas en el ámbito del reconocimiento facial. FaceNet utiliza una estrategia basada en embeddings faciales, generando una representación numérica única para cada rostro. Esta arquitectura ofrece altos niveles de precisión y es ampliamente utilizada en sistemas biométricos; sin embargo, requiere procesos adicionales para comparar y clasificar los rostros, además de una implementación más compleja.

También se analizó YOLO Face, una adaptación de la familia YOLO orientada a la detección facial. Esta arquitectura destaca por su velocidad y capacidad para detectar múltiples rostros en tiempo real. Sin embargo, su enfoque principal es la localización de rostros dentro de una imagen, por lo que para realizar reconocimiento e identificación sería necesario complementar la solución con modelos adicionales.

Finalmente, se evaluó MobileNetV2, una red neuronal convolucional diseñada específicamente para dispositivos móviles y sistemas con recursos limitados. Esta arquitectura se caracteriza por su bajo consumo de memoria, rapidez de procesamiento y facilidad de integración con TensorFlow Lite, lo que la convierte en una alternativa atractiva para aplicaciones móviles que requieren realizar inferencias localmente.

Después de analizar las características de cada arquitectura, se seleccionó MobileNetV2 como base para el desarrollo del proyecto. La decisión se tomó considerando el tamaño relativamente reducido del dataset, la necesidad de ejecutar el modelo directamente en un dispositivo móvil y la facilidad para convertirlo e integrarlo en una aplicación desarrollada con Flutter. Aunque arquitecturas como FaceNet o Yolo Face pueden alcanzar mayores niveles de precisión en escenarios de gran escala, MobileNetV2 ofrece un mejor equilibrio entre rendimiento, eficiencia y complejidad de implementación para los objetivos planteados.

Por estas razones, **MobileNetV2** fue considerada la alternativa más adecuada para el contexto educativo del proyecto, permitiendo desarrollar un sistema funcional de reconocimiento facial capaz de ejecutarse de manera eficiente en dispositivos móviles sin depender de servicios externos o procesamiento en la nube.



–Img 4. Arquitectura de procesamiento MobilNetV2

5.3 Configuración y entrenamiento del Modelo

Una vez finalizada la etapa de preparación del dataset y seleccionada la arquitectura MobileNetV2, se procedió al entrenamiento del modelo de reconocimiento facial utilizando TensorFlow y Google Colab. Esta etapa tuvo como objetivo permitir que la red neuronal aprendiera a identificar las características distintivas de cada estudiante a partir de las imágenes faciales procesadas previamente.

Como parte del preprocesamiento, todas las imágenes fueron redimensionadas a una resolución de 160×160 píxeles, compatible con la arquitectura seleccionada. Posteriormente, los valores de los píxeles fueron normalizados para facilitar el proceso de aprendizaje y mejorar la estabilidad del entrenamiento.

Con el fin de evaluar la capacidad de generalización del modelo, el dataset fue dividido en dos subconjuntos: un conjunto de entrenamiento, utilizado para ajustar los parámetros de la red neuronal, y un conjunto de validación, empleado para medir el desempeño del modelo sobre imágenes que no habían sido utilizadas durante el aprendizaje. Adicionalmente, se aplicaron técnicas de aumento de datos (Data Augmentation), incluyendo variaciones de brillo, rotaciones, desplazamientos y zoom, con el propósito de incrementar la diversidad de ejemplos disponibles y reducir el riesgo de sobreajuste.

```
{
  "20170638": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "20170639": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "21170495": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "21170937": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "22170618": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "22170621": {
    "train": 21,
    "val": 5,
    "total": 26
  },
  "22170640": {
    "train": 21,
    "val": 5,
    "total": 26
  }
}
```

–Img 5. División del Dataset para entrenamiento y validación

El entrenamiento se realizó utilizando el optimizador Adam y la función de pérdida Categorical Crossentropy, configuraciones ampliamente utilizadas en problemas de clasificación multiclase. Asimismo, se definieron hiperparámetros como la tasa de aprendizaje, el tamaño de lote (batch size) y el número máximo de épocas de entrenamiento. Durante este proceso también se emplearon mecanismos de monitoreo que permitieron reducir automáticamente la tasa de aprendizaje cuando el modelo dejaba de mostrar mejoras significativas y detener el entrenamiento anticipadamente en caso de detectar convergencia.

```
Epoch 19/30  
10/10 — 6s 602ms/step - accuracy: 0.9932 - loss: 0.0305 - val_accuracy: 1.0000 - val_loss: 0.0027 - learning_rate: 3.0000e-04  
Epoch 20/30  
10/10 — 5s 472ms/step - accuracy: 1.0000 - loss: 0.0155 - val_accuracy: 1.0000 - val_loss: 0.0026 - learning_rate: 3.0000e-04  
Epoch 21/30  
10/10 — 7s 737ms/step - accuracy: 1.0000 - loss: 0.0238 - val_accuracy: 1.0000 - val_loss: 0.0025 - learning_rate: 9.0000e-05  
Epoch 22/30  
10/10 — 8s 469ms/step - accuracy: 0.9932 - loss: 0.0352 - val_accuracy: 1.0000 - val_loss: 0.0024 - learning_rate: 9.0000e-05  
Epoch 23/30  
10/10 — 8s 815ms/step - accuracy: 1.0000 - loss: 0.0175 - val_accuracy: 1.0000 - val_loss: 0.0023 - learning_rate: 9.0000e-05  
Epoch 24/30  
10/10 — 8s 525ms/step - accuracy: 0.9864 - loss: 0.0348 - val_accuracy: 1.0000 - val_loss: 0.0022 - learning_rate: 2.7000e-05  
Epoch 25/30  
10/10 — 7s 746ms/step - accuracy: 0.9932 - loss: 0.0228 - val_accuracy: 1.0000 - val_loss: 0.0022 - learning_rate: 2.7000e-05  
Épocas ejecutadas: 25  
Último accuracy: 0.9931972622871399  
Último val_accuracy: 1.0
```

—Img 6. Entrenamiento del Modelo

Una vez concluido el entrenamiento, el modelo fue evaluado mediante métricas de clasificación como Accuracy, Precision, Recall y F1-Score, obteniendo resultados satisfactorios para las condiciones planteadas en el proyecto. Finalmente, el modelo entrenado fue convertido al formato TensorFlow Lite (.tflite), lo que permitió su integración dentro de la aplicación móvil desarrollada en Flutter y su ejecución directamente en el dispositivo sin necesidad de conexión a servicios externos.

5.4 Resultados y Validación del Modelo

Una vez finalizado el entrenamiento, se procedió a evaluar el desempeño del modelo utilizando el conjunto de validación. Para ello se emplearon métricas comúnmente utilizadas en problemas de clasificación multiclase, entre las que destacan Accuracy, Precision, Recall y F1-Score.

Resultados

Métrica	Valor Obtenido
Loss	0.0022
Accuracy	100.00%
Precision	100.00%
Recall	100.00%
F1-Score	100.00%

Estos resultados indican que el modelo fue capaz de clasificar correctamente todas las imágenes presentes en el conjunto de validación, obteniendo un desempeño sobresaliente dentro de las condiciones evaluadas. Asimismo, los valores de Precision y Recall muestran que el modelo logró identificar correctamente cada una de las clases sin generar errores de clasificación dentro del conjunto utilizado para la evaluación.

Además de las métricas obtenidas durante la validación, se realizaron pruebas prácticas utilizando imágenes diferentes a las empleadas durante el entrenamiento. Estas pruebas permitieron verificar el funcionamiento del sistema integrado dentro de la aplicación móvil, observando que el modelo era capaz de reconocer correctamente a los estudiantes registrados y recuperar su información académica desde la base de datos local.

Sin embargo, es importante considerar que los resultados deben analizarse en el contexto del tamaño del dataset utilizado. Debido a que el conjunto de datos fue relativamente pequeño y estuvo conformado por un número limitado de estudiantes, las métricas obtenidas no garantizan que el modelo mantenga exactamente el mismo nivel de desempeño en escenarios de mayor escala o con condiciones significativamente diferentes a las presentes durante la recopilación de datos.

A pesar de esta limitación, los resultados obtenidos demuestran que el proceso de construcción del dataset, preprocesamiento de imágenes, selección de la arquitectura y entrenamiento del modelo fue adecuado para los objetivos planteados en el proyecto. El sistema logró reconocer correctamente a los estudiantes registrados y ejecutar las inferencias directamente en un dispositivo móvil, validando la viabilidad de utilizar técnicas de visión artificial y Machine Learning para aplicaciones de reconocimiento facial en entornos educativos.

5.5 Desarrollo e Implementación de la Aplicación Móvil

Una vez entrenado y validado el modelo de reconocimiento facial, se procedió al desarrollo de una aplicación móvil que permitiera utilizarlo de manera práctica dentro de un entorno educativo. Para ello se empleó Flutter como framework de desarrollo, debido a su capacidad para crear aplicaciones multiplataforma mediante una única base de código y su compatibilidad con TensorFlow Lite.

La aplicación fue diseñada bajo una arquitectura modular, separando la lógica de reconocimiento facial, la administración de datos y la interfaz de usuario en diferentes componentes. Esto permitió mejorar la organización del proyecto y facilitar futuras modificaciones o ampliaciones del sistema.



–Img 7. Arquitectura de la aplicación

El modelo entrenado fue convertido al formato TensorFlow Lite (.tflite) e integrado dentro de la aplicación mediante la librería tflite_flutter. De esta manera, las inferencias se realizan directamente en el dispositivo móvil, evitando la necesidad de utilizar servicios externos o conexiones a internet para el reconocimiento de estudiantes.

Para la detección de rostros se utilizó MediaPipe Face Detection, encargado de localizar automáticamente la región facial dentro de las imágenes capturadas por la cámara. Una vez detectado el rostro, este es recortado, redimensionado y procesado antes de ser enviado al modelo de inteligencia artificial para su clasificación.

La aplicación incorpora un sistema de autenticación básica para restringir el acceso a las funcionalidades administrativas. Después de iniciar sesión, el usuario puede acceder a un menú principal desde el cual es posible utilizar el módulo de reconocimiento facial o consultar el panel de control.

El módulo de reconocimiento facial permite capturar una imagen mediante la cámara del dispositivo, identificar al estudiante y mostrar información almacenada en una base de datos SQLite, incluyendo nombre, número de control, semestre y carrera. Además, cada reconocimiento exitoso genera automáticamente un registro de asistencia que almacena la fecha, hora y nivel de confianza de la predicción.

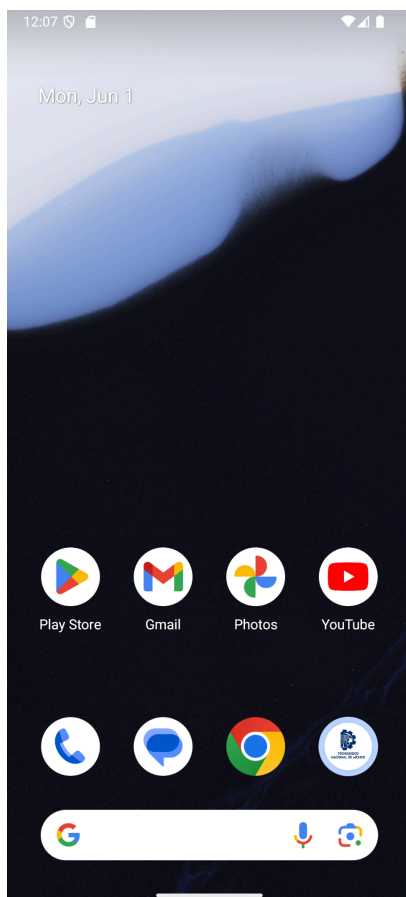
Por otra parte, el panel de control permite visualizar la información de los estudiantes registrados, actualizar sus datos académicos y consultar el historial de asistencias generadas por el sistema. Estas funcionalidades convierten la aplicación en una herramienta capaz de demostrar de manera práctica el uso de técnicas de visión artificial y aprendizaje automático en un contexto educativo.

La integración de Flutter, TensorFlow Lite, MediaPipe y SQLite permitió desarrollar una solución completamente funcional que ejecuta todas las tareas localmente en el dispositivo móvil, garantizando tiempos de respuesta rápidos y reduciendo la dependencia de servicios externos.

5.6 Capturas de Pantalla de Ejecución

5.6.1 Interfaz del Móvil

En esta imagen podemos apreciar el icono de la App el cual es el logo del TECNM.



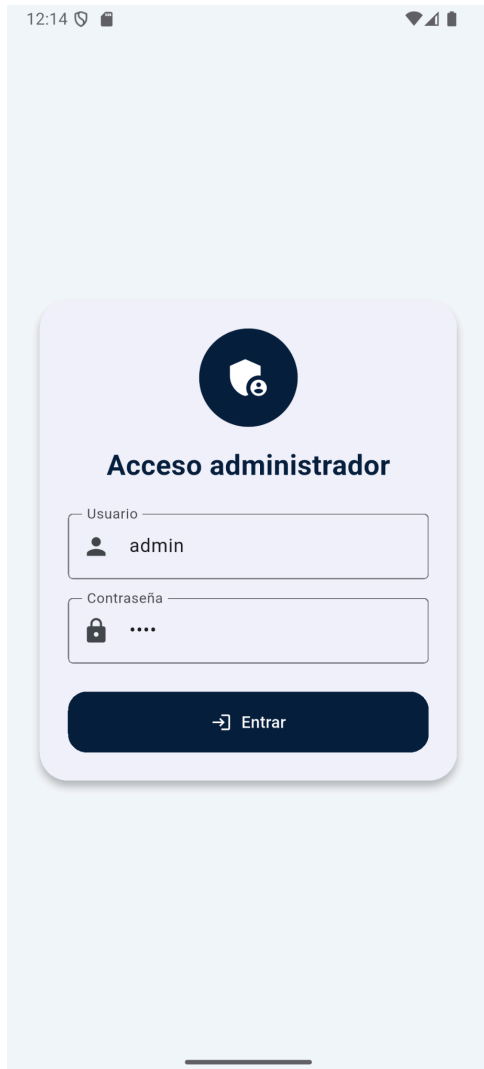
5.6.2 Pantalla de carga con el nombre de la App y Slogan

En esta imagen vemos la pantalla de carga de cuando arranca la aplicacion con su logo y su eslogan principal



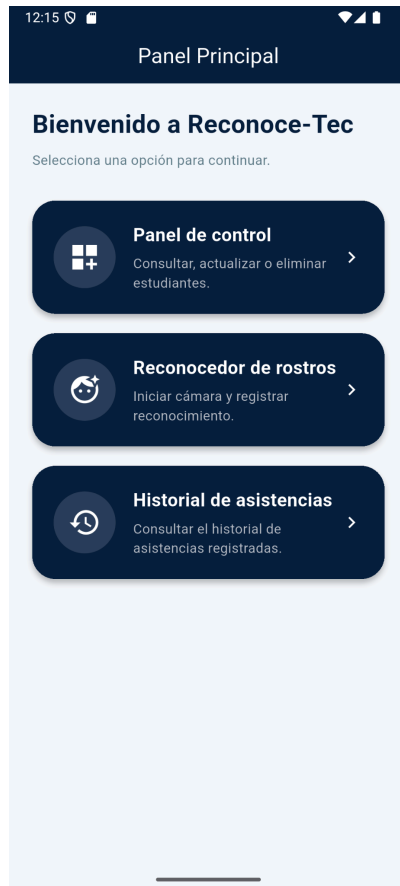
5.6.3 Login

En esta imagen vemos el login donde solamente las credenciales de administrador son permitidas



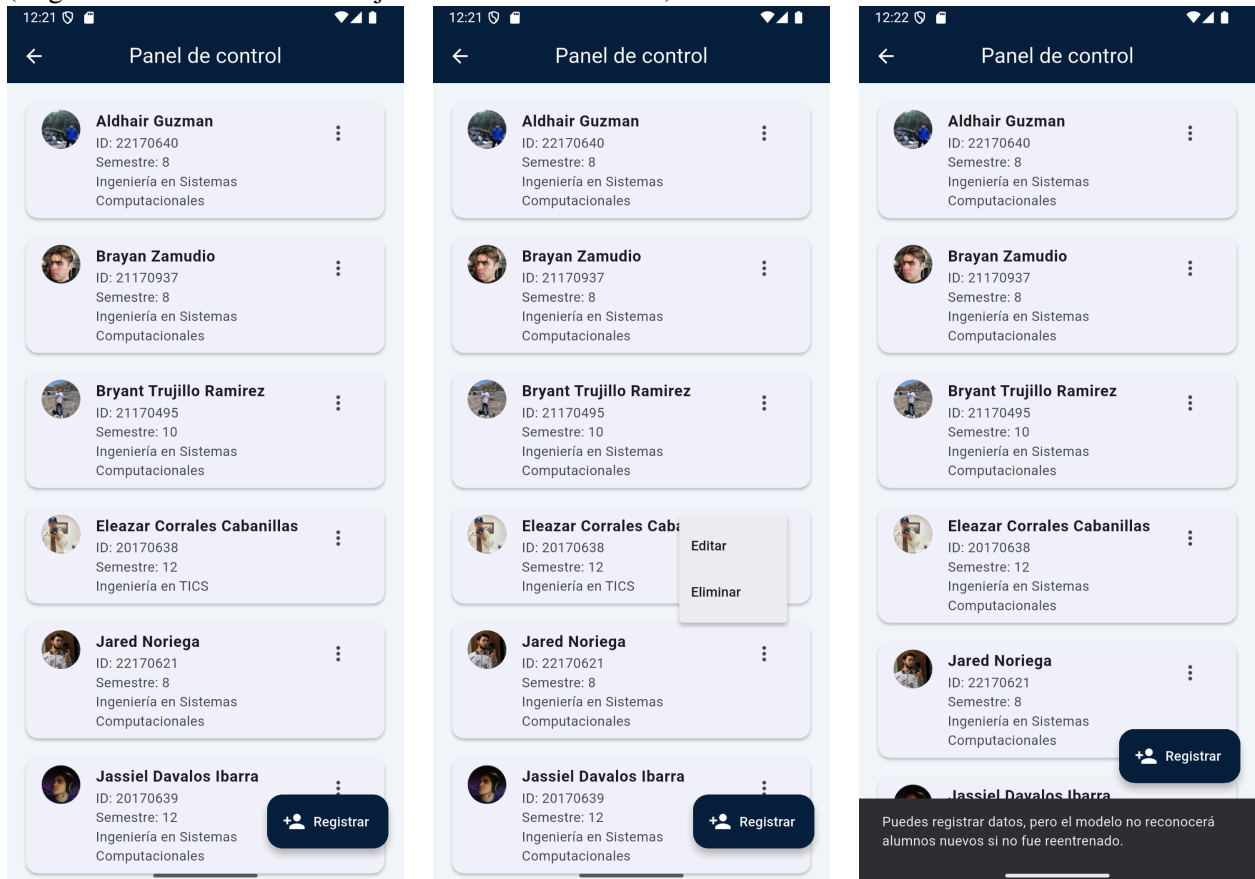
5.6.4 Panel principal de la Aplicación

En esta imagen podemos apreciar el panel principal de la aplicación así como las funcionalidades con las que cuenta, siendo cada una de ellas botones accionables que dirigen al usuario a cada una de las herramientas



5.6.5 Panel de control de alumnos

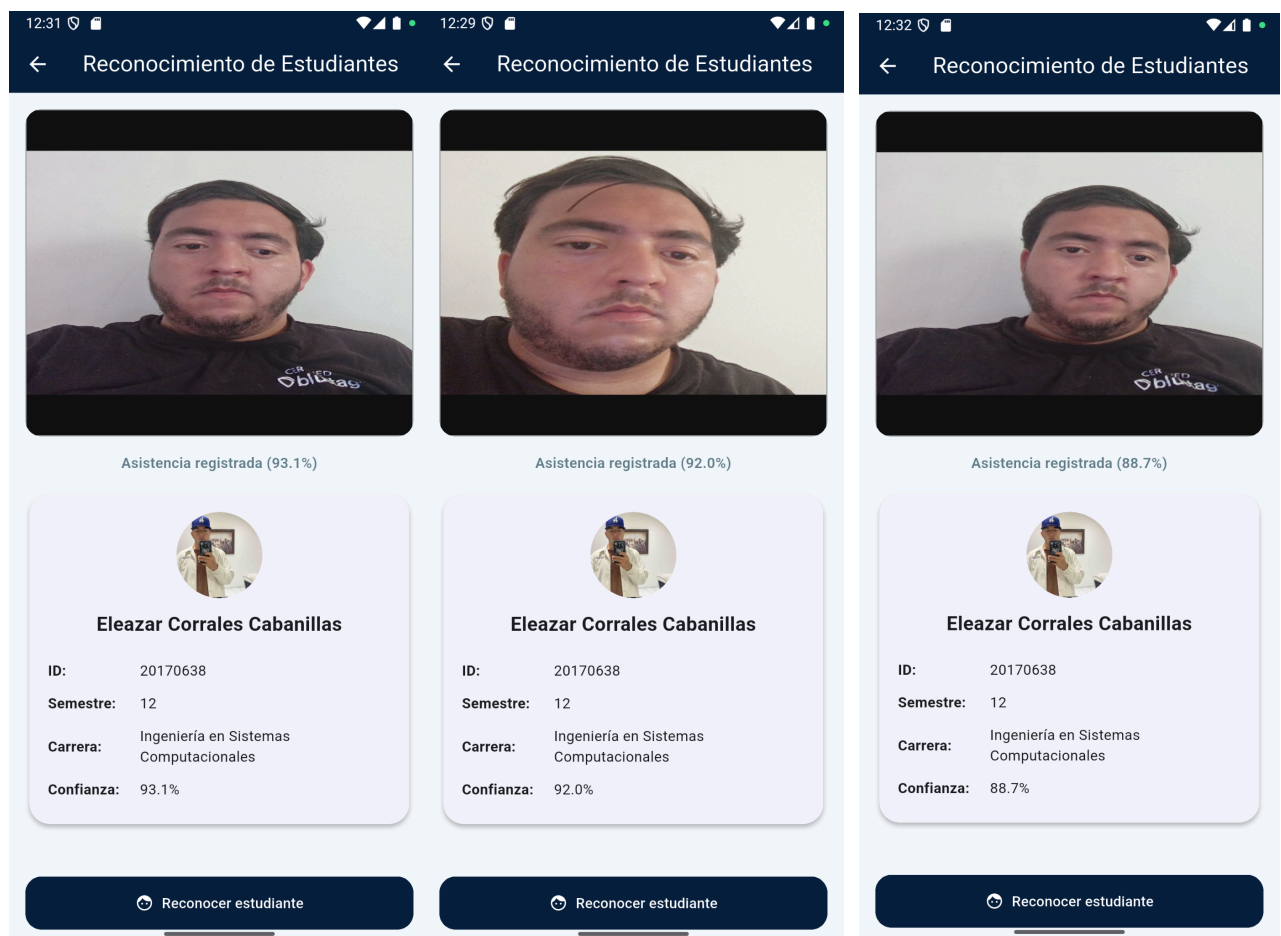
En estas imágenes apreciamos el panel de control donde podemos validar cada una de sus funciones, consultar la información de los estudiantes, editar la información de estudiantes y así como el concepto de otras funcionalidades que se pudieran añadir como eliminar estudiantes y registrar otros estudiantes (Registrar un estudiante no re-ajusta el modelo utilizado).



5.6.6 Reconocedor de Rostros

Aquí podemos visualizar la funcionalidad principal de nuestra aplicación donde a través de la cámara podemos capturar una imagen y predecir cuál de los alumnos es, debajo de esto lo que se hace es la captura, la normalización de la imagen, la detección del rostro, la predicción y ya con eso podemos llenar el card de información del alumno siempre y cuando sea uno de los alumnos que participaron en el entrenamiento.

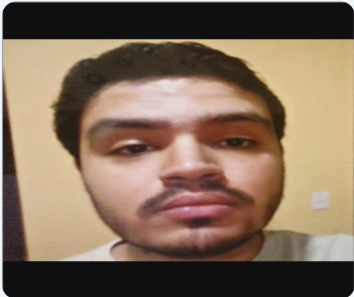
Nota: Si la precisión se < 60 lo marca como estudiante no reconocido dado su baja similitud con la predicción del modelo.



5.6.7 Pruebas

12:42 3G

← Reconocimiento de Estudiantes



Asistencia registrada (93.8%)



Bryant Trujillo Ramirez

ID: 21170495
Semestre: 10
Carrera: Ingeniería en Sistemas Computacionales
Confianza: 93.8%

Reconocer estudiante

12:47 3G

← Reconocimiento de Estudiantes



Estudiante no reconocido



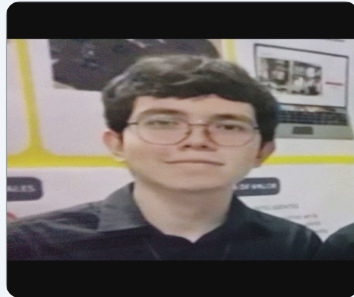
Sin estudiante reconocido

ID: --
Semestre: --
Carrera: --
Confianza: --

Reconocer estudiante

12:42 3G

← Reconocimiento de Estudiantes



Asistencia registrada (80.8%)



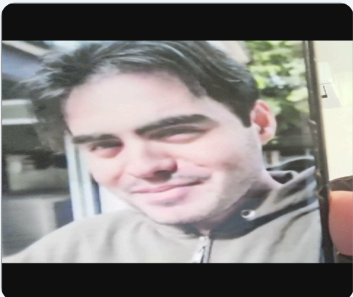
Omar Langerica

ID: 22170618
Semestre: 8
Carrera: Ingeniería en Sistemas Computacionales
Confianza: 80.8%

Reconocer estudiante

12:44 3G

← Reconocimiento de Estudiantes



Asistencia registrada (96.9%)



Brayan Zamudio

ID: 21170937
Semestre: 8
Carrera: Ingeniería en Sistemas Computacionales
Confianza: 96.9%

Reconocer estudiante

12:45 3G

← Reconocimiento de Estudiantes



Asistencia registrada (85.8%)



Jared Noriega

ID: 22170621
Semestre: 8
Carrera: Ingeniería en Sistemas Computacionales
Confianza: 85.8%

Reconocer estudiante

12:46 3G

← Reconocimiento de Estudiantes



Asistencia registrada (98.6%)



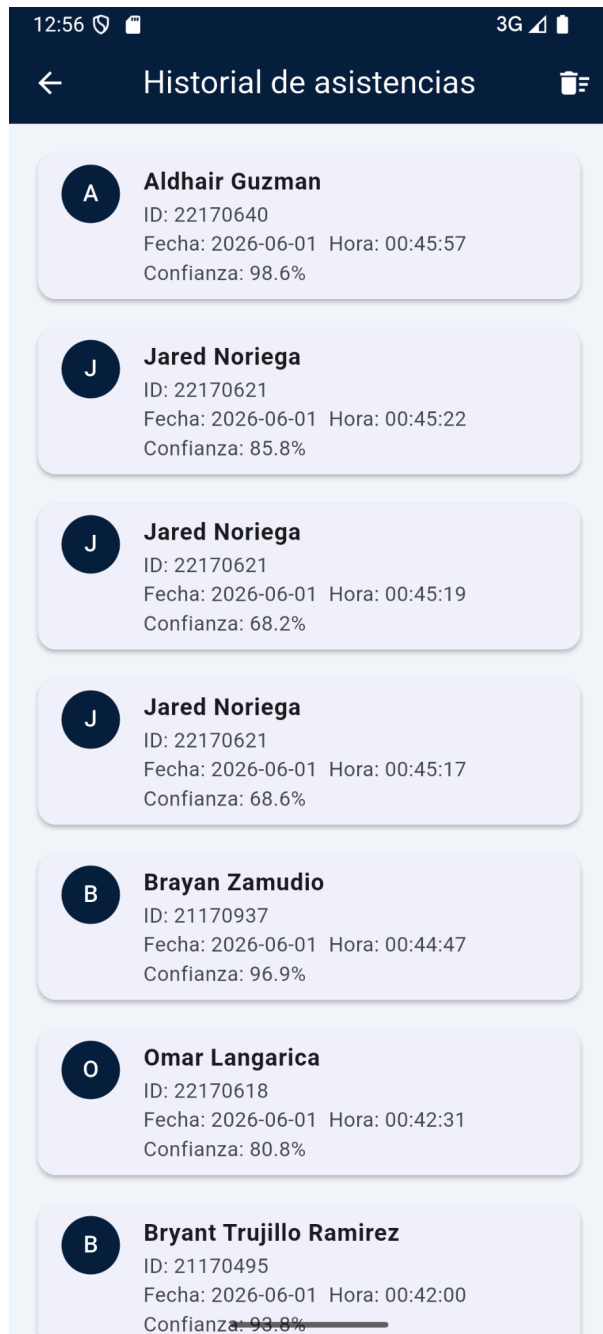
Aldhair Guzman

ID: 22170640
Semestre: 8
Carrera: Ingeniería en Sistemas Computacionales
Confianza: 98.6%

Reconocer estudiante

5.6.8 Registro de asistencia

Por último, pusimos en práctica este proyecto llevándolo más allá y desarrollando un sistema de asistencia que una vez que el Estudiante es reconocido por el modelo de IA, este se registra en una tabla de la BD con sus datos y la fecha y hora de registro de asistencia. Esto lo cual sería muy útil de utilizar en escuelas de diferentes niveles académicos.



Historial de asistencias				
A	Aldhair Guzman	ID: 22170640	Fecha: 2026-06-01	Hora: 00:45:57 Confianza: 98.6%
J	Jared Noriega	ID: 22170621	Fecha: 2026-06-01	Hora: 00:45:22 Confianza: 85.8%
J	Jared Noriega	ID: 22170621	Fecha: 2026-06-01	Hora: 00:45:19 Confianza: 68.2%
J	Jared Noriega	ID: 22170621	Fecha: 2026-06-01	Hora: 00:45:17 Confianza: 68.6%
B	Brayan Zamudio	ID: 21170937	Fecha: 2026-06-01	Hora: 00:44:47 Confianza: 96.9%
O	Omar Langerica	ID: 22170618	Fecha: 2026-06-01	Hora: 00:42:31 Confianza: 80.8%
B	Bryant Trujillo Ramirez	ID: 21170495	Fecha: 2026-06-01	Hora: 00:42:00 Confianza: 93.8%

6. Resumen de Tecnologías Utilizadas

Para el desarrollo del proyecto se utilizaron diversas herramientas y tecnologías orientadas al procesamiento de imágenes, entrenamiento de modelos de inteligencia artificial, desarrollo móvil y gestión de datos.

- **Flutter**
Framework de desarrollo multiplataforma utilizado para la construcción de la aplicación móvil. Permitió desarrollar una interfaz moderna e intuitiva utilizando una única base de código.
- **Dart**
Lenguaje de programación empleado por Flutter para implementar la lógica de negocio, navegación entre pantallas y funcionalidades generales de la aplicación.
- **TensorFlow**
Framework de Machine Learning utilizado para el entrenamiento, validación y evaluación del modelo de reconocimiento facial.
- **TensorFlow Lite**
Versión optimizada de TensorFlow para dispositivos móviles. Permitió ejecutar el modelo de inteligencia artificial directamente en el dispositivo sin depender de servidores externos.
- **OpenCV**
Biblioteca utilizada para el procesamiento de imágenes y la extracción automática de fotogramas a partir de los videos empleados en la construcción del dataset.
- **MediaPipe Face Detection**
Herramienta de visión artificial utilizada para detectar y localizar rostros dentro de las imágenes, permitiendo recortar automáticamente la región facial antes del entrenamiento y durante las inferencias en la aplicación.
- **Google Colab**
Entorno de desarrollo basado en la nube utilizado para el entrenamiento del modelo, aprovechando recursos computacionales superiores a los disponibles localmente.
- **SQLite**
Sistema de gestión de bases de datos local empleado para almacenar la información académica de los estudiantes y los registros de asistencia generados por la aplicación.
- **VSCode** Entorno de desarrollo utilizado para la construcción, compilación y pruebas de la aplicación móvil en dispositivos Android y emuladores.

La integración de estas tecnologías permitió desarrollar una solución completa que combina visión artificial, aprendizaje automático, desarrollo móvil y almacenamiento local de datos dentro de una única aplicación funcional.

7. Conclusiones

El desarrollo de este proyecto permitió aplicar de manera práctica los conocimientos adquiridos en las áreas de visión artificial, Machine Learning y desarrollo de aplicaciones móviles. A través de la construcción de un dataset propio, el procesamiento de imágenes, la investigación de arquitecturas neuronales y el entrenamiento de un modelo de inteligencia artificial, fue posible comprender las diferentes etapas involucradas en la creación de un sistema de reconocimiento facial.

La investigación realizada sobre distintas arquitecturas de reconocimiento facial permitió identificar las ventajas y limitaciones de cada alternativa. A partir de este análisis se seleccionó MobileNetV2 debido a su equilibrio entre precisión, velocidad y consumo de recursos, características especialmente importantes para su ejecución en dispositivos móviles mediante TensorFlow Lite.

Los resultados obtenidos durante la validación mostraron un desempeño satisfactorio dentro de las condiciones del proyecto, logrando identificar correctamente a los estudiantes incluidos en el conjunto de datos y demostrando la viabilidad de integrar modelos de inteligencia artificial dentro de aplicaciones móviles. Asimismo, la incorporación de una base de datos SQLite permitió complementar el reconocimiento facial con funcionalidades prácticas como la consulta de información académica y el registro automático de asistencia.

Es importante reconocer que el tamaño del dataset utilizado fue limitado en comparación con los conjuntos de datos empleados en aplicaciones comerciales. Sin embargo, para los fines académicos del proyecto, resultó suficiente para demostrar el proceso completo de diseño, entrenamiento, validación e implementación de un sistema de visión artificial funcional. Como trabajo futuro, sería conveniente ampliar el número de estudiantes, incrementar la cantidad de imágenes por clase y explorar arquitecturas especializadas como FaceNet o ArcFace para mejorar la capacidad de generalización del sistema.

En conclusión, se logró desarrollar una aplicación móvil funcional capaz de detectar y reconocer estudiantes mediante técnicas de visión artificial, integrando de forma exitosa herramientas de inteligencia artificial, procesamiento de imágenes, bases de datos y desarrollo móvil en una solución orientada al contexto educativo.

Referencias

- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. <https://arxiv.org/abs/1801.04381>
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. <https://arxiv.org/abs/1503.03832>
- Wang, M., & Deng, W. (2021). Deep Face Recognition: A Survey. <https://doi.org/10.1016/j.neucom.2020.10.081>
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach. <https://aima.cs.berkeley.edu/>
- OpenAI. (2026). ChatGPT (GPT-5.5) [Large Language Model]. <https://chatgpt.com/>
- Google. (2025). Gemini 3.5 Pro. <https://gemini.google.com/>